

CS 3110 HW4

1) This is the pseudocode required for this question:

```
public int CountShortestPaths(Map<String, List<String>> adjList, String u,
String v) {
    /* TODO */
    Map<String,Integer> distance = new HashMap<>(); // O(1)
    Map<String,Integer> count = new HashMap<>(); //O(1)
    Queue<String> queue = new LinkedList<>();//O(1)
    queue.add(u);//O(1)
    distance.put(u,0) //O(1)
    count.put(u,1)//O(1)
    queue.add(u)//O(1)
    while (!queue.isEmpty()) { //O(n)
        String a = queue.poll();
        List<String> neighbors = adjList.get(a);
        for(int i = 0; i<neighbors.size(); i++){ //O(m)
            if (!distance.containsKey(x)) { // All operations in the if condition
are are O(1)
                distance.add(x);
                distance.put(x, distance.get(u) + 1);
                queue.add(v);
            }
            else if (distance.get(x)==distance.get(u)+1){ // All operations are
O(1)
                count.put(x,count.get(u)+count.get(w))
            }
        }
        return count.getDefault(v,0); //To avoid errors is v is not reachable
from u or if the graph is empty.O(1) time complexity
    }
}
```

Over here are using the BFS algorithm to calculate the total number of shortest paths from one vertex to another because it follows layer by layer exploration where each vertex is visited once and each edge is at most visited twice.

All initializations, additions and putting processes follow $O(1)$ time complexity, so we are only left with the time complexities of the inner for loop and the outer while loop. Let us assume that $n = V(\text{vertices})$ and $m = E(\text{edges})$, then we can observe that the inner for loop travels through all the neighbors of a vertex, so we can that its time complexity is $O(E)$ and the outer while loop travels through all the vertices in the list so its time complexity is $O(V)$, hence the total time complexity of this algorithm will be $O(V+E)$.

2) Pseudocode:

```
public boolean FindThePath(Map<String, List<String>> adjList) {  
3)    // Step 1: Reverse the graph  
4)    Map<String, List<String>> reverse = new HashMap<>(); //O(1)  
5)    List<String> vertex = new ArrayList<>(adjList.keySet()); //O(1)  
6)    for (int i = 0; i < vertex.size(); i++) { // O(n)  
7)        String u = vertex.get(i);  
8)        if (!reverse.containsKey(u)) {  
9)            reverse.put(u, new ArrayList<>());  
10)        }  
11)  
12)  
13)        List<String> n= adjList.get(u);  
14)        for (int j = 0; j < n.size(); j++) { //O(m)  
15)            String v = n.get(j);  
16)            if (!reverse.containsKey(v)) {  
17)                reverse.put(v, new ArrayList<>());  
18)                reverse.get(v).add(u); // The edge gets reversed  
19)            }  
20)        }  
21)    for (int i = 0; i < vertex.size(); i++) { //O(n)  
22)        String candidate = vertex.get(i);  
23)        Set<String> visited = new HashSet<>();  
24)        dfs(candidate, visited, reverse);  
25)        if (visited.size() == adjList.size()) {  
26)            return true;  
27)        }  
28)    }  
29)    return false;  
30) }  
31) void dfs(String node, Set<String> visited, Map<String, List<String>> r) {  
32)    visited.add(node); //O(n)  
33)    List<String> neighbors;  
34)    if (r.containsKey(node)) {  
35)        neighbors = r.get(node);  
36)    } else {  
37)        neighbors = new ArrayList<>();  
38)    }  
39)  
40)    for (int i = 0; i < neighbors.size(); i++) { //O(m)  
41)        String neighbor = neighbors.get(i);
```

```

42)     if (!visited.contains(neighbor)) {
43)         dfs(neighbor, visited, r);
44)     }
45) }
46) }

```

Now, let's try and analyze the runtime of this algorithm. Let us assume that $n = V(\text{vertices})$ and $m = E(\text{edges})$. As we can see that all the initializations and code under the if statements take $O(1)$ time, so we can focus on the loops. Let's try and break down our pseudocode in parts to make our work easy.

```

47) for (int i = 0; i < vertex.size(); i++) { // O(n)
48)     String u = vertex.get(i);
49)     if (!reverse.containsKey(u)) {
50)         reverse.put(u, new ArrayList<>());
51)     }
52)
53)
54)     List<String> n= adjList.get(u);
55)     for (int j = 0; j < n.size(); j++) { //O(m)
56)         String v = n.get(j);
57)         if (!reverse.containsKey(v)) {
58)             reverse.put(v, new ArrayList<>());
59)             reverse.get(v).add(u); // The edge gets reversed
60)         }
61)     }

```

This is the first for loop in our code. We can see that there is an inner for loop which does m iterations since it's iterating through all the neighbors of a node, so we can say that its time complexity is $O(m)$, whereas the outer for loop runs n iterations, so its run time complexity is $O(n)$. Thus the total runtime for this loop is $O(n+m)$.

```

62) for (int i = 0; i < vertex.size(); i++) { //O(n)
63)     String candidate = vertex.get(i);
64)     Set<String> visited = new HashSet<>();
65)     dfs(candidate, visited, reverse);
66)     if (visited.size() == adjList.size()) {
67)         return true;
68)     }
69) }
70) return false;
71) }

```

This is the second for loop used in the code. Since it iterates through each and every vertex we can say that, we can say it iterates n times, however, I have used a helper method in my code for doing the dfs, so we also need to go through the helper method's runtime complexity to get the worst case runtime for this algorithm.

```
72) void dfs(String node, Set<String> visited, Map<String, List<String>> r) {
73)     visited.add(node); //O(n)
74)     List<String> neighbors;
75)     if (r.containsKey(node)) {
76)         neighbors = r.get(node);
77)     } else {
78)         neighbors = new ArrayList<>();
79)     }
80)
81)     for (int i = 0; i < neighbors.size(); i++) { //O(m)
82)         String neighbor = neighbors.get(i);
83)         if (!visited.contains(neighbor)) {
84)             dfs(neighbor, visited, r);
85)         }
86)     }
87) }
```

This is the helper method that I have used. Since we are going to add n nodes, we can say that the runtime for adding all the nodes in this case would be $O(n)$. Also, there is a for loop that has been used in this method which iterates through all the neighbors of a node and makes m iterations, so we can say that the runtime for this loop is $O(m)$ and hence the total runtime for this method will be $O(n+m)$.

Since, this method was used in a for loop which had n iterations runtime, we can say that the worst case runtime for the whole loop will be $O(n*(n+m))$.

Hence, we can say that the worst case runtime for this algorithm will be $O(n*(n+m))$ since $n*(n+m) \gg (n+m)$ (the worst case runtime of the first for loop).

3) Pseudocode for this question:

```

public static Map<String, String> isTwoColorable(
    Map<String, List<String>> adjList,
    String u,
    String v
) {
    Map<String, String> color = new HashMap<>(); //O(1)
    List<String> nodes = new ArrayList<>(adjList.keySet()); //O(1)

    for (int i = 0; i < nodes.size(); i++) { //O(n)
        String startNode = nodes.get(i); //O(n)

        if (!color.containsKey(startNode)) { //O(n+m)
            Queue<String> queue = new LinkedList<>(); //O(1)
            queue.add(startNode); //O(1)
            color.put(startNode, u); //O(1)

            while (!queue.isEmpty()) { //O(n)
                String currentNode = queue.poll();
                String currentNodeColor = color.get(currentNode);

                String nextColor;
                if (currentNodeColor.equals(u)) {
                    nextColor = v;
                } else {
                    nextColor = u;
                }

                List<String> neighbors = adjList.get(currentNode);
                for (int j = 0; j < neighbors.size(); j++) { //O(m)
                    String neighbor = neighbors.get(j);

                    if (!color.containsKey(neighbor)) { //O(1)
                        color.put(neighbor, nextColor);
                        queue.add(neighbor);
                    } else if (color.get(neighbor).equals(currentNodeColor)) {
                        //O(1)
                        return null; // if the graph is not 2-colorable
                    }
                }
            }
        }
    }

    return color;
}

```

Let us analyze the worst case time complexity of this algorithm. Let us assume that $n = V(\text{vertices})$ and $m = E(\text{edges})$. As we can see that all the initializations and code under the if statements take $O(1)$ time, so we can focus on the loop. Over here, I have performed the BFS algorithm inside an if statement so that it

only runs for the nodes that are not present in the hashmap color, which means that the BFS algorithm wouldn't take place for all the nodes and hence we can observe that the outer for loop iterates n times so that means its runtime is $O(n)$ and the runtime for the BFS algorithm is $O(n+m)$, since the while iterates through all the nodes and the for loop in the algorithm iterates through all the edges. Since, $O(n) \ll O(n+m)$, we can say that the worst case time complexity for this algorithm will be $O(n+m)$.

4)

Initial d value	∞	∞	∞	∞	∞	∞	∞	∞
iteration	A	B	C	D	E	F	G	H
1	0	1	∞	5	∞	∞	∞	∞
2	0	1	3	5	10	∞	∞	∞
3	0	1	3	4	10	5	∞	8
4	0	1	3	4	10	5	∞	8
5	0	1	3	4	10	5	∞	6
6	0	1	3	4	10	5	∞	6
7	0	1	3	4	10	5	∞	6